

机电系统仿真复习

一些函数

eig: 返回特征值

inv: 求逆矩阵

nargin: 获取函数输入参数的数目

nargout: 获取函数输出参数的数目

conv: 求卷积或多项式乘法 $\text{conv}([1,0,1], [2,7]) = (2,7,2,7)$

polyval: 用于多项式计算 eg. $y = x^2 + 2x + 1$, 求 $y(2)$, 则 $x=[1,2,1]$, polyval(x,2)

rand(): 用于生成(0,1)均匀分布的随机数, rand(m, n)则是生成元素大小在(0,1)的随机数 $m \times n$ 矩阵

如果想生成(a,b)范围内的就在这个基础上操作, 进行放大平移, 先放大 $(b-a) * \text{rand}(m, n)$, 这样范围就变成了(0,b-a), 接下来再进行平移操作, 移动一个a, 也就是 $r = a + (b-a) * \text{rand}(m, n)$, 此时r便是(a,b)范围内的 $m \times n$ 矩阵

diff: 求导数, diff(y,t)就是 $\frac{dy}{dt}$, diff(y,t,2)就是 $\frac{d^2y}{dt^2}$

pzmap: 用来绘制零极点图 (p就是peak极点, z就是zero零点)

clf: 清空图窗 (clear figure)

clear: 清空保存的变量

clc: 清空命令行 (clear command)

length(x): 返回数组的最大维度的长度

三维曲线图: plot3

三维网线图: mesh(X,Y,Z)

三维曲面图: surf(X,Y,Z)

除上面外可以再了解一些后面几章用到的函数, 去年有考到

微分方程求解内容

先给出微分方程的描述 (定义函数), 一般要化成一阶微分的形式

```

$$\left\{ \begin{array}{l} \ddot{y} - (1 - y^2)\dot{y} + y = 0 \\ y(0) = 0, \dot{y}(0) = 1 \end{array} \right.$$

```

先定义 $y_1 = y_2$, 所以初始条件 $y_1 = 0, y_2 = 1$, 方程就化成的 $\dot{y}_2 = (1 - y_1^2)y_2 - y_1$

然后就是定义函数(这里把 y_1, y_2 都放到一个数组y里了y(1)就代表 y_1)

```
function dydt = vanderpol(t, y, mu)
    dydt = [y(2); (1 - y(1)^2)*y(2) - y(1)];
end
```

给出初始条件 `y0=[0;1]` (注意是`;`)

给出时间区间 `tspan = [0 20];`

最后求解，画图

```
[t, y] = ode45(@vanderpol(t, y), tspan, y0);  
plot(t, y(:, 1));
```

求稳态误差

求稳态误差可以用 `dcgain(ess)`，其中 `ess=s* sys *u`，`sys`是系统传递函数，`u`由输入决定，eg.斜坡输入 $u = \frac{1}{s^2}$

利用开环传递函数产生的曲线来分析系统稳定性和相关指标

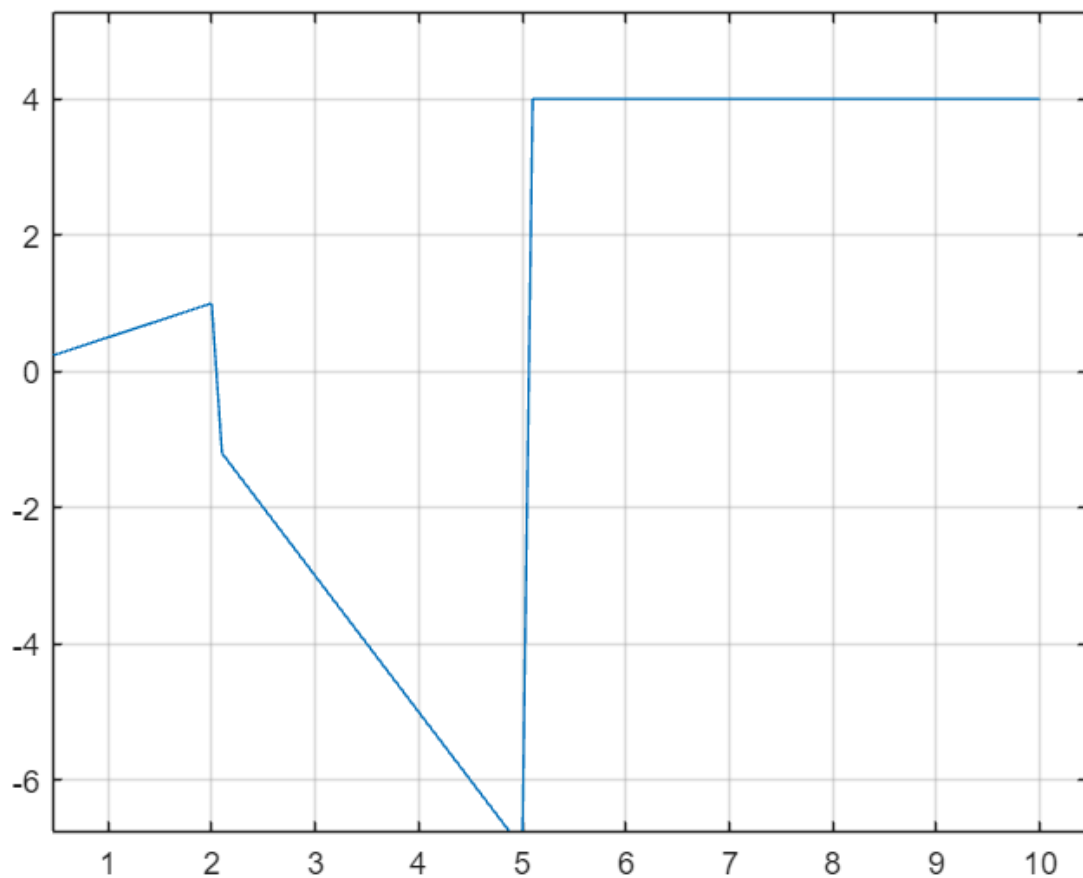
只有在求系统响应曲线才是用闭环传递函数

分段函数

之前期末考试那个分段绘图如果没记错的话就是这个类型

编写一个函数，使其能够产生如下的分段函数：并调用此函数绘制 `x=0:0.1:10` 对应的 `y(x)` 曲线，其中 `y(x)` 定义为 `Erroneous nesting of equation structures`

```
% chapter4_3.m  
%% 分段函数  
% code by wisezenn  
function y = chapter4_3(x)  
    y = zeros(size(x));  
    y(x <= 2) = 0.5 .* x(x <= 2);  
    y(x > 2 & x <= 5) = 3 - 2 .* x(x > 2 & x <= 5);  
    y(x > 5) = 4 .* ones(size(x(x > 5)));  
end  
  
% 函数调用  
x = 0:0.1:10;  
% 调用函数计算y的值  
y = chapter4_3(x);  
% 绘制曲线  
plot(x, y);  
grid on;
```



求阶乘

```
% factorial_sum.m
%% 阶乘累加函数，输入参数为n，将计算1!+2!+.....+n!
% code by wisezenn
function factorial_sum(n)
    sum = 0;
    i = 1;
    while i <= n
        factorial = 1;
        j = 1;
        while j <= i
            factorial = factorial * j;
            j = j + 1;
        end
        sum = sum + factorial;
        i = i + 1;
    end
    disp(sum)

% 函数调用
%% Chapter4_1 阶乘累加
```

```
% 调用函数计算1!+2!+.....+10!
```

```
factorial_sum(10)
```

```
% 运行结果
```

```
4037913
```

冒泡排序

```
% BubbleSort.m
```

```
%% 冒泡排序
```

```
% code by wisezenn
```

```
function xSorted=BubbleSort(x)
```

```
x_len=length(x);
```

```
for i=1:x_len-1
```

```
    for j=1:x_len-i
```

```
        if(x(j)>x(j+1))
```

```
            [x(j),x(j+1)]=swap(x(j),x(j+1));
```

```
        end
```

```
    end
```

```
disp([num2str(i),'.Sort:x=',num2str(x)]);%将每次排序结果输出
```

```
end
```

```
xSorted=x;
```

```
end
```

```
%% 交换函数
```

```
function [a,b]=swap(x,y)
```

```
a=y;
```

```
b=x;
```

```
end
```

```
% 函数调用
```

```
%% Chapter4_2 冒泡排序
```

```
X=[25,4,8,2,3,1,9,0,10,3,5];
```

```
disp(['Before Sort:X=',num2str(X)]);
```

```
disp('-----');
```

```
y=BubbleSort(X);%调用BubbleSort
```

```
disp(['Bubble Sort:x=',num2str(y)]);%输出最后结果
```

```
% 运行结果
```

```
Before Sort:X=25 4 8 2 3 1 9 0 10 3 5
```

```
-----
```

```
1.Sort:x=4 8 2 3 1 9 0 10 3 5 25
```

```
2.Sort:x=4 2 3 1 8 0 9 3 5 10 25
```

```
3.Sort:x=2 3 1 4 0 8 3 5 9 10 25
```

```
4.Sort:x=2 1 3 0 4 3 5 8 9 10 25
```

```
5.Sort:x=1 2 0 3 3 4 5 8 9 10 25
```

```
6.Sort:x=1 0 2 3 3 4 5 8 9 10 25
```

```
7.Sort:x=0 1 2 3 3 4 5 8 9 10 25
```

```
8.Sort:x=0 1 2 3 3 4 5 8 9 10 25
```

```
9.Sort:x=0 1 2 3 3 4 5 8 9 10 25
```

```
10.Sort:x=0 1 2 3 3 4 5 8 9 10 25
```

```
Bubble Sort:x=0 1 2 3 3 4 5 8 9 10 25
```

